

Claude Conversations

Claude Code Session

write and run a program that goes through, and puts ALL my prompts I have sent claude code for this project in a single text file. Just my prompts. Include this one.

hmm it looks like not all my prompts are there, and also some things there are responses rather than prompts.

"C:\Users\Luk27182\claude\projects\C--Users-Luk27182-Desktop-NL2SPICE\d864e0b3-4c8d-48a2-ba53-883223f7a739\subagents\agent-a7646656953eb13ec.jsonl" also include prompts from here

read through this folder, and understand each topology for each type of filter. You need to design system prompts for each type of filter/topology: In the system prompts, you should describe ALL relevant equations needed to create a filter with a given topology and requirements. In total, this will be 16 prompts. Also include 3 worked examples for each prompt, including a final output spice netlist. Include what things to change if things don't work as expected. Don't touch any of the existing files, make a separate folder for your work. Include an isolated "base prompt" that describes general output formatting.

adjust it a bit: 1. it actually shouldn't be so explicitly a system prompt. More of an "explainer" on how to solve this type of circuit topology. 2. double check EVERY prompt you are using by making up some specs, following the instructions you have to generate a netlist, and then using the util functions to test them the pass/attenuated frequencies.. Come back to me if anything fails.

one thing- make sure to only run one test at a time. Whenever you run a test, my PC slows down. So for example, don't run ALL tests when you just want to see if notch pass is working.

are you sure that your tests follow the instructions exactly as in the educational discriptions? double check each and walk through them. Get back to me on each.

in all 16, replace "circuit diagrams" with example netlists with unspecified values for resistance/capacitance.

now look at the file system: I added a New_Datagen folders with files of the form "checkpoint_{something}.json". The json has rows for "prompt" as well as "toplogy". Your goal is to create a single, new json called "RAG_files.json" where you attach prompts from these checpoint jsons to their corresponding "filter_system_prompt" files. To do so, you must look at the type of filter (eg low pass) as well as the topology (eg rc_single). You should look at what topologies exist in each checkpoint json to ensure nothing is missed.

continue

Explore the project at C:\Users\Luk27182\Desktop\NL2SPICE thoroughly. I need to understand:

1. The overall folder structure
2. All filter types and topologies present (read every relevant file)
3. Any existing system prompts or prompt templates
4. Any SPICE netlist examples
5. Any datasets or documentation describing the filter parameters/equations

Read all key files — especially any Python scripts, datasets (CSV/JSON), and documentation. Report back with: the full

directory tree, each filter type and topology found, any equations or parameters mentioned, and any example SPICE netlists. Be very thorough.

dB/decade and RC filter capabilities

describe what is meant by a db/decade. What is possible for a single RC filter?

will this code ensure that the db/decade is actually within the ranges? check all cases.

adjust the generation instead so that it first samples/stores a random attenuation (just in db/decade) that is within the valid ranges. Then, in generating the rest, it converts that into actual db changed based on the frequencies it sampled (calculating the number of decades).

now make a new version that copies the structure but works for high pass filters.

now do the same thing for band pass filters

finally, do notch filter.

```
Directory: C:\Users\Luk27182\Desktop\NL2SPICE
Mode LastWriteTime Length Name
-----
d---- 4/13/2026 4:27 PM datagen
Directory: C:\Users\Luk27182\Desktop\NL2SPICE\datagen
Mode LastWriteTime Length Name
-----
-a---- 4/13/2026 4:15 PM 44475 bandpass.py
-a---- 4/13/2026 4:15 PM 12113 bpf_dataset.json
-a---- 4/13/2026 4:10 PM 36773 highpass.py
-a---- 4/13/2026 4:11 PM 10061 hpf_dataset.json
-a---- 4/13/2026 4:04 PM 36190 lowpass.py
-a---- 4/13/2026 4:05 PM 10016 lpf_dataset.json
-a---- 4/13/2026 4:27 PM 48764 notchpass.py
PS C:\Users\Luk27182\Desktop\NL2SPICE>
```

this is my folder structure. Now make another file called "totalgen.py" that calls each of the py files with the same arguments and concatenates all the data json files into one giant file.

SPICE simulation in Python with conda

How to simulate SPICE in python? conda package

show how to use PySPICE example for a low pass filter

Collecting pypspice

Downloading PySpice-1.5-py2.py3-none-any.whl.metadata (15 kB)

Requirement already satisfied: PyYAML>=5.3 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (6.0.3)

Requirement already satisfied: cffi>=1.14 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (2.0.0)

Requirement already satisfied: matplotlib>=3.2 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (3.10.6)

Requirement already satisfied: numpy>=1.18 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (2.2.6)

Collecting ply>=3.11 (from pypspice)

Downloading ply-3.11-py2.py3-none-any.whl.metadata (844 bytes)

Requirement already satisfied: scipy>=1.4 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (1.16.2)

Requirement already satisfied: requests>=2.23 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from pypspice) (2.32.5)

Requirement already satisfied: pycparser in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from cffi>=1.14->pypspice) (2.23)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (1.3.1)

Requirement already satisfied: cycycler>=0.10 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (1.4.8)

Requirement already satisfied: packaging>=20.0 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (25.0)

Requirement already satisfied: pillow>=8 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (12.0.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (3.2.0)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from matplotlib>=3.2->pypspice) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.2->pypspice) (1.17.0)

Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from requests>=2.23->pypspice) (3.4.3)

Requirement already satisfied: idna<4,>=2.5 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from requests>=2.23->pypspice) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from requests>=2.23->pypspice) (2.6.3)

Requirement already satisfied: certifi>=2017.4.17 in c:\users\luk27182\miniconda3\envs\hope\lib\site-packages (from requests>=2.23->pypspice) (2026.1.4)

Downloading PySpice-1.5-py2.py3-none-any.whl (158 kB)

Downloading ply-3.11-py2.py3-none-any.whl (49 kB)

Installing collected packages: ply, pypspice

```

----- 1/2 [pypspice]
----- 1/2 [pypspice]
----- 2/2 [pypspice]

```

Successfully installed ply-3.11 pypspice-1.5

OSError Traceback (most recent call last)

Cell In[28], line 16

13 circuit.C(1, 'out', circuit.gnd, 1@u_uF)

15 # Run transient simulation

---> 16 simulator = circuit.simulator(temperature=25, nominal_temperature=25)

17 analysis = simulator.transient(step_time=1@u_us, end_time=5@u_ms)

19 # Plot results

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\Netlist.py:1292, in Circuit.simulator(self, *args, **kwargs)

1291 def simulator(self, *args, **kwargs):

-> 1292 return CircuitSimulator.factory(self, *args, **kwargs)

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\Simulation.py:1167, in CircuitSimulator.factory(cls, circuit, *args, **kwargs)

1164 kwargs['parallel'] = True

1166 if sub_cls is not None:

-> 1167 return sub_cls(circuit, *args, **kwargs)

1168 else:

1169 raise ValueError('Unknown simulator type')

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\NgSpice\Simulation.py:99, in NgSpiceSharedCircuitSimulator.__init__(self, circuit, **kwargs)

97 ngspice_shared = kwargs.get('ngspice_shared', None)

98 if ngspice_shared is None:

---> 99 self._ngspice_shared = NgSpiceShared.new_instance()

100 else:

101 self._ngspice_shared = ngspice_shared

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\NgSpice\Shared.py:449, in NgSpiceShared.new_instance(cls, ngspice_id, send_data, verbose)

447 else:

448 cls._logger.debug("New instance for id {}".format(ngspice_id))

--> 449 instance = cls(ngspice_id=ngspice_id, send_data=send_data, verbose=verbose)

450 cls._instances[ngspice_id] = instance

451 return instance

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\NgSpice\Shared.py:479, in NgSpiceShared.__init__(self, ngspice_id, send_data, verbose)

476 self._extensions = []

478 self._library_path = None

--> 479 self._load_library(verbose)

480 self._init_ngspice(send_data)

482 self._is_running = False

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\PySpice\Spice\NgSpice\Shared.py:539, in NgSpiceShared._load_library(self, verbose)

537 if verbose:

538 print(message)

--> 539 self._ngspice_shared = ffi.dlopen(self.library_path)

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\cffi\api.py:150, in FFI.dlopen(self, name, flags)

147 raise TypeError("dlopen(name): name must be a file name, None, "

148 "or an already-opened 'void ** handle")

149 with self._lock:

--> 150 lib, function_cache = _make_ffi_library(self, name, flags)

151 self._function_caches.append(function_cache)

152 self._libraries.append(lib)

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\cffi\api.py:834, in _make_ffi_library(ffi, libname, flags)

832 def _make_ffi_library(ffi, libname, flags):

833 backend = ffi._backend

--> 834 backendlib = _load_backend_lib(backend, libname, flags)

835 #

836 def accessor_function(name):

File c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\cffi\api.py:829, in _load_backend_lib(backend, name, flags)

827 if first_error is not None:

828 msg = "%s. Additionally, %s" % (first_error, msg)

```
--> 829 raise OSError(msg)
830 return backend.load_library(path, flags)
OSError: cannot load library 'c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-
packages\PySpice\Spice\NgSpice\Spice64_dll\dl-vs\ngspice.dll': error 0x7e. Additionally, ctypes.util.find_library() did not
manage to locate a library called 'c:\\Users\\Luk27182\\miniconda3\\envs\\hope\\Lib\\site-
packages\\PySpice\\Spice\\NgSpice\\Spice64_dll\\dl-vs\\ngspice.dll'
```

full script to create the env and install things.

High-impedance sensor low-pass filter design

```
```json
```

A high-impedance sensor signal requires heavy low-pass filtering before digitisation. Signals up to 1 kHz must pass with  $\leq 1.0$  dB loss. Signals at 5 kHz must be attenuated by at least 55.9 dB. Allow one op-amp to buffer the source; keep remaining components passive (R and C only).

Provide the design as a SPICE netlist. The following node names are mandatory for automated simulation:

- Input node : VIN
- Output node : VOUT
- Ground / 0 V : GND

Name all resistors with the prefix R (e.g. R1, R2), all capacitors with the prefix C (e.g. C1, C2), and any op-amp with the designator U1. Include a voltage source V1 connected between VIN and GND. End the netlist with a .END statement. Do not include a simulation command (.AC, .TRAN, etc.) in the netlist; the simulation will be configured externally.

```
```
```

python how to simulate this circuit where I set the input frequencies

I want to directly put in a spice netlist and run the spice software (I have it installed)

Q: Which SPICE software do you have?

A: I'm not sure how to check

C:\Users\Luk27182\Downloads\ngspice-46_64\Spice64\lib\ngspice

```
-----
Ngspice Info
-----
```

```
Fatal error in SPICE
-----
```

```
OK
-----
```

Running ngspice... Could not parse output. Raw ngspice output:

An exception has occurred, use %tb to see the full traceback. SystemExit

```
c:\Users\Luk27182\miniconda3\envs\hope\Lib\site-packages\IPython\core\interactiveshell.py:3707: UserWarning: To exit: use
'exit', 'quit', or Ctrl-D. warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)
```

```
import subprocess
```

```
import tempfile
```

```
import os
```

```
NGSPICE_EXE = r"C:\Users\Luk27182\Downloads\ngspice-46_64\Spice64\bin\ngspice.exe"
```

```
NETLIST = """\
```

```
* 5th-Order Butterworth Low-Pass Filter
```

```
V1 VIN 0 AC 1
```

```

E_U1 N1 0 VIN 0 1
R1 N1 N2 10000
R2 N2 N3 10000
R3 N3 N4 10000
R4 N4 N5 10000
R5 N5 VOUT 10000
C1 N2 0 8.66E-9
C2 N3 0 22.6E-9
C3 N4 0 28.0E-9
C4 N5 0 22.6E-9
C5 VOUT 0 8.66E-9
.options noacct
.AC DEC 100 10 100000
.PRINT AC VM(VOUT)
.END

```

```

with tempfile.NamedTemporaryFile(mode="w", suffix=".cir",
delete=False, dir=tempfile.gettempdir()) as f:
f.write(NETLIST)
netlist_path = f.name
result = subprocess.run(
[NGSPICE_EXE, "-b", netlist_path],
capture_output=True, text=True
)
os.unlink(netlist_path)
print("=== STDOUT ===")
print(repr(result.stdout)) # repr() shows invisible characters and line endings
print()
print("=== STDERR ===")
print(repr(result.stderr))
print()
print("Return code:", result.returncode)

```

```

=== STDOUT ===
"
=== STDERR ===
"
Return code: 0

```

```

=== ngspice output ===
Note: No compatibility mode selected!
Circuit: * 5th-order butterworth low-pass filter
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000
Using SPARSE 1.3 as Direct Linear Solver
Note: v1: has no value, DC 0 assumed
No. of Data Rows : 401
* 5th-order butterworth low-pass filter
AC Analysis Tue Apr 14 14:40:46 2026

-----
Index frequency vm(vout)
-----
0 1.000000e+01 9.896259e-01
1 1.023293e+01 9.891448e-01

```

2 1.047129e+01 9.886418e-01
3 1.071519e+01 9.881158e-01
4 1.096478e+01 9.875660e-01
5 1.122018e+01 9.869912e-01
6 1.148154e+01 9.863903e-01
7 1.174898e+01 9.857623e-01
8 1.202264e+01 9.851060e-01
9 1.230269e+01 9.844201e-01
10 1.258925e+01 9.837034e-01
11 1.288250e+01 9.829546e-01
12 1.318257e+01 9.821723e-01
13 1.348963e+01 9.813550e-01
14 1.380384e+01 9.805014e-01
15 1.412538e+01 9.796100e-01
16 1.445440e+01 9.786790e-01
17 1.479108e+01 9.777070e-01
18 1.513561e+01 9.766922e-01
19 1.548817e+01 9.756329e-01
20 1.584893e+01 9.745273e-01
21 1.621810e+01 9.733736e-01
22 1.659587e+01 9.721697e-01
23 1.698244e+01 9.709139e-01
24 1.737801e+01 9.696039e-01
25 1.778279e+01 9.682378e-01
26 1.819701e+01 9.668134e-01
27 1.862087e+01 9.653285e-01
28 1.905461e+01 9.637807e-01
29 1.949845e+01 9.621678e-01
30 1.995262e+01 9.604874e-01
31 2.041738e+01 9.587370e-01
32 2.089296e+01 9.569142e-01
33 2.137962e+01 9.550164e-01
34 2.187762e+01 9.530410e-01
35 2.238721e+01 9.509853e-01
36 2.290868e+01 9.488467e-01
37 2.344229e+01 9.466224e-01
38 2.398833e+01 9.443096e-01
39 2.454709e+01 9.419056e-01
40 2.511886e+01 9.394075e-01
41 2.570396e+01 9.368125e-01
42 2.630268e+01 9.341177e-01
43 2.691535e+01 9.313202e-01
44 2.754229e+01 9.284171e-01
45 2.818383e+01 9.254056e-01
46 2.884032e+01 9.222827e-01
47 2.951209e+01 9.190457e-01
Frequency Gain (dB) Pass/Fail

35 Hz -0.98 dB PASS
35 Hz -0.98 dB PASS
35 Hz -0.98 dB PASS
35 Hz -0.98 dB FAIL
35 Hz -0.98 dB FAIL
35 Hz -0.98 dB FAIL

=== ngspice output ===

Note: No compatibility mode selected!

Circuit: * 5th-order butterworth low-pass filter

Doing analysis at TEMP = 27.000000 and TNOM = 27.000000

Using SPARSE 1.3 as Direct Linear Solver

Note: v1: has no value, DC 0 assumed

No. of Data Rows : 401

* 5th-order butterworth low-pass filter

AC Analysis Tue Apr 14 14:42:09 2026

Index frequency vm(vout)

| | | |
|----|--------------|--------------|
| 0 | 1.000000e+01 | 9.896259e-01 |
| 1 | 1.023293e+01 | 9.891448e-01 |
| 2 | 1.047129e+01 | 9.886418e-01 |
| 3 | 1.071519e+01 | 9.881158e-01 |
| 4 | 1.096478e+01 | 9.875660e-01 |
| 5 | 1.122018e+01 | 9.869912e-01 |
| 6 | 1.148154e+01 | 9.863903e-01 |
| 7 | 1.174898e+01 | 9.857623e-01 |
| 8 | 1.202264e+01 | 9.851060e-01 |
| 9 | 1.230269e+01 | 9.844201e-01 |
| 10 | 1.258925e+01 | 9.837034e-01 |
| 11 | 1.288250e+01 | 9.829546e-01 |
| 12 | 1.318257e+01 | 9.821723e-01 |
| 13 | 1.348963e+01 | 9.813550e-01 |
| 14 | 1.380384e+01 | 9.805014e-01 |
| 15 | 1.412538e+01 | 9.796100e-01 |
| 16 | 1.445440e+01 | 9.786790e-01 |
| 17 | 1.479108e+01 | 9.777070e-01 |
| 18 | 1.513561e+01 | 9.766922e-01 |
| 19 | 1.548817e+01 | 9.756329e-01 |
| 20 | 1.584893e+01 | 9.745273e-01 |
| 21 | 1.621810e+01 | 9.733736e-01 |
| 22 | 1.659587e+01 | 9.721697e-01 |
| 23 | 1.698244e+01 | 9.709139e-01 |
| 24 | 1.737801e+01 | 9.696039e-01 |
| 25 | 1.778279e+01 | 9.682378e-01 |
| 26 | 1.819701e+01 | 9.668134e-01 |
| 27 | 1.862087e+01 | 9.653285e-01 |
| 28 | 1.905461e+01 | 9.637807e-01 |
| 29 | 1.949845e+01 | 9.621678e-01 |
| 30 | 1.995262e+01 | 9.604874e-01 |
| 31 | 2.041738e+01 | 9.587370e-01 |
| 32 | 2.089296e+01 | 9.569142e-01 |
| 33 | 2.137962e+01 | 9.550164e-01 |
| 34 | 2.187762e+01 | 9.530410e-01 |
| 35 | 2.238721e+01 | 9.509853e-01 |
| 36 | 2.290868e+01 | 9.488467e-01 |
| 37 | 2.344229e+01 | 9.466224e-01 |
| 38 | 2.398833e+01 | 9.443096e-01 |
| 39 | 2.454709e+01 | 9.419056e-01 |
| 40 | 2.511886e+01 | 9.394075e-01 |
| 41 | 2.570396e+01 | 9.368125e-01 |


```

42 2.630268e+01 9.341177e-01
43 2.691535e+01 9.313202e-01
44 2.754229e+01 9.284171e-01
45 2.818383e+01 9.254056e-01
46 2.884032e+01 9.222827e-01
47 2.951209e+01 9.190457e-01
Frequency Gain (dB) Pass/Fail

```

```

35 Hz -0.98 dB PASS
35 Hz -0.98 dB PASS
35 Hz -0.98 dB PASS
35 Hz -0.98 dB FAIL
35 Hz -0.98 dB FAIL
35 Hz -0.98 dB FAIL

```

Parsed 401 data points. Freq range: 10.0 – 100000 Hz
Frequency Gain (dB) Pass/Fail

```

100 Hz -5.00 dB FAIL
501 Hz -19.67 dB FAIL
1000 Hz -30.20 dB FAIL
1995 Hz -45.39 dB FAIL
5012 Hz -74.48 dB PASS
10000 Hz -101.61 dB PASS

```

why are frequencies not exactly what I specified?

Passive anti-aliasing filter design

```

```json

```

A simple anti-aliasing filter is required. It must pass signals at 20 kHz with less than 1.0 dB attenuation while rejecting frequencies at 60 kHz by at least 9.5 dB. The circuit must use only passive components and should be as low-cost as possible.

Provide the design as a SPICE netlist. The following node names are mandatory for automated simulation:

- Input node : VIN
- Output node : VOUT
- Ground / 0 V : GND

Name all resistors with the prefix R (e.g. R1, R2), all capacitors with the prefix C (e.g. C1, C2), and any op-amp with the designator U1. Include a voltage source V1 connected between VIN and GND. End the netlist with a .END statement. Do not include a simulation command (.AC, .TRAN, etc.) in the netlist; the simulation will be configured externally

```

```

```

But isn't the problem statement asking for a 20db/decade falloff?

adjust this code to use your test

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

```

19953 Hz -0.91 dB >= -1.0 dB PASS
60256 Hz -4.95 dB <= -9.5 dB FAIL

```

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

19953 Hz -15.44 dB >= -1.0 dB FAIL
60256 Hz -24.99 dB <= -9.5 dB PASS

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

19953 Hz -33.96 dB >= -1.0 dB FAIL
60256 Hz -43.56 dB <= -9.5 dB PASS

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

19953 Hz -0.91 dB >= -1.0 dB PASS
60256 Hz -4.95 dB <= -9.5 dB FAIL

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

19953 Hz -4.36 dB >= -1.0 dB FAIL
60256 Hz -13.23 dB <= -9.5 dB PASS

Parsed 301 data points. Freq range: 1000.0 – 1000000 Hz
Frequency Gain (dB) Limit Pass/Fail

19953 Hz -1.87 dB >= -1.0 dB FAIL
60256 Hz -8.02 dB <= -9.5 dB FAIL

Why is this so hard? Is it that the frequencies are so close together?

in real world uses, how far apart are cutoff/stop? Is it a constant offset or constant ratio?

is GSM harder than 96KHz ADC?

how to determine when a single RC filter is enough? I know it gives 20db/decade, but it takes some time to get to that slope (smooth transition), so sometimes the 20db/decade is not achievable for a small number of decades (<1)

do something like this to do a one stage RC filter. Plot the PREDICTED attenuation function $A(f)$ as well as measured simulation values

how does it work then for two stage filter? Attenuation as function of the input frequency

So if its entirely constant, what is the rolloff in db/decade

give the code to run a simple low pass filter again

explain each part of that netlist

adjust to simulation of 5 level RC filters (no buffer)

Building an invisible harp with laser and phototransistor

10PCS 3 Pins KY-008 650nm Dot Diode Copper Head Laser Sensor Transmitter Module Red 5V Compatible with Arduino

HiLetgo 20pcs 5MM Photo Transistor Phototransistor PT334-6C Photo Transistor Phototransistor Round F5 3V

can these two parts be used to make an invisible harp?

what parts do I need for sound/

8002A Audio Amplifier and Speaker Module, Plug-and-Play DAC Speaker, 3W Mono Class D Power Amp Board, 90dB SNR, Compatible with ESP32, Raspberry Pi, Arduino

how about this? it has no amazon reviews

what if I just want buzzes?

do I need an amplifier for a buzzer?

I want to play multiple notes at once using mosi library

SPICE netlist attenuation calculator

Using this, make a minimal function that takes in a spice netlist (just the connections, eg something like

```
.param R=1k.param C=1nV1 VIN 0 AC 1R1 VIN N1 {R}C1 N1 0 {C}R2 N1 N2 {R}C2 N2 0 {C}R3 N2 N3 {R}C3 N3 0 {C}R4 N3 N4 {R}C4 N4 0 {C}R5 N4 VOUT {R}C5 VOUT 0 {C}
```

as well as an input frequency for VIN (sep parameter to the function.) And it will return attenuation at VOUT. If nodes vin/vout are not defined or the simulation fails, return NONE

predict attenuation here- with formulas.

at the moment for the code as given, -4.304656677743172

```
```python
netlist = """
.param R=1k
.param C=1n
V1 VIN 0 AC 1
R1 VIN N1 {R}
C1 N1 0 {C}
R2 N1 N2 {R}
C2 N2 0 {C}
R5 N2 VOUT {R}
C5 VOUT 0 {C}
"""

db = simulate_attenuation(netlist, frequency=40e3)
...

```

here isnt f 40k?

## Converting multi-stage RC filter to single stage

```

python
netlist = """
.param R=1k
.param C=1n
V1 VIN 0 AC 1
R1 VIN N1 {R}
C1 N1 0 {C}
R2 N1 N2 {R}
C2 N2 0 {C}
R5 N2 VOUT {R}
C5 VOUT 0 {C}
"""

db = simulate_attenuation(netlist, frequency=40e3)
print(db) # e.g. -18.4

```

make this a simple single stage RC filter.

use formulas to figure out what the true rolloff should be?

```

python
def simulate_attenuation(netlist_body: str, frequency: float) -> float | None:
 """
 Run an ngspice AC simulation and return the attenuation at VOUT (in dB).

 Parameters

 netlist_body : str
 SPICE netlist fragment containing only the circuit elements and .param
 lines (no title line, no .AC/.PRINT/.END). Must define nodes VIN and VOUT.
 frequency : float
 Frequency (Hz) at which to evaluate the gain/attenuation.

 Returns

 float | None
 Attenuation in dB at the requested frequency, or None on any failure.
 """

```

assume I have a function like this and a netlist. Write python code that, if given a netlist, will figure out the "cutoff frequency" given jsut the netlist by sampling points.

will this work for what type of filters?

make it more clear its specifically for low pass.

full code.

also make it return a plot of all samples, a line through it, and a line at the cutoff frequency.

make it faster- does not have to find the exact frequency, just be close enough.

### Three stage passive filter design

```
``python
netlist = """
.param R=1k
.param C=1n
V1 VIN 0 AC 1
R1 VIN VOUT {R}
C1 VOUT 0 {C}
"""
...

```

make this into a 3 stage passive filter.

what is cutoff frequency here?

### Parameterized RC filter netlist template

write a python string with a netlist for a single stage RC filter: make it formattable to put in whatever R/C values I want.

```
``python
netlist = """\
* Single-Stage RC Low-Pass Filter
* R = {R}, C = {C}

V1 IN 0 AC 1
R1 IN OUT {R}
C1 OUT 0 {C}
"""
...

```

copy this format (no commands or .END).

Make a dict where keys are things like "single\_stage\_rc\_lowpass" and the values are the netlist.

keys:

single stage rc lowpass

two stage rc lowpass

buffered single stage rc lowpass (with op amp)

buffered (at the start) single stage rc low pass

make sure IN and OUT are always called "VIN" and "VOUT"

format it well for ease of reading/editing

## Organizing filter utilities in code

suppose I also will be adding high pass/band pass/notch filters. If I do so, which parts of this to add to an utils? How to organize

## Simplifying output consistency

simplify this as much as possible: Ensure outputs are the saem.

## Buffered RC low-pass filter with op-amp

```
```python
netlist = """
.param R=1k
.param C=1n

* 3-Stage Passive RC Low-Pass Filter
* fc = 1 / (2 * pi * R * C) ≈ 159 kHz per stage

V1 VIN 0 AC 1

* Stage 1
R1 VIN N1 {R}
C1 N1 0 {C}

* Stage 2
R2 N1 N2 {R}
C2 N2 0 {C}

* Stage 3
R3 N2 VOUT {R}
C3 VOUT 0 {C}
"""

db = simulate_attenuation(netlist, frequency=40e3)
print(db) # e.g. -18.4
```
```

copy this format to make the filter buffered (at the input) via opp-amp

where are you making the op amp?

thats ok! Ebuf is perfect.

```
from dataclasses import dataclass, field
from typing import Dict, Set
from pathlib import Path# -----# Template
Definition# -----@dataclassclass NetlistTemplate:
 name: str
 template: str
 required_params: Set[str] = field(
 default_factory=set
)
 def render(self, **params) -> str:
 missing = self.required_params - params.keys()
 if missing:
 raise ValueError(
 f"[{self.name}] Missing parameters: {sorted(missing)}"
)
 return self.template.format(**params)# -----
--# Library / Registry# -----class NetlistLibrary:
 def __init__(self):
 self._templates: Dict[str, NetlistTemplate] = {}
 def register(self, key: str, template: NetlistTemplate):
 if key in self._templates:
 raise ValueError(f"Template '{key}' already exists")
 self._templates[key] = template
 def get(self, key: str) -> NetlistTemplate:
 if key not in self._templates:
 raise KeyError(f"Unknown template: {key}")
 return self._templates[key]
 def render(self, key: str, **params) -> str:
 return self.get(key).render(**params)# -----# Optional: File Loader# -----def
```

```
load_template_from_file(path: str, name: str, required_params: Set[str]): content = Path(path).read_text() return
NetlistTemplate(name=name, template=content, required_params=required_params)# -----# Define
Templates# -----def build_library() -> NetlistLibrary: lib = NetlistLibrary()
```

adjust this to create filters for:

single stage RC

multi stage RC

single stage RC where is an op-amp buffer at the start

multi stage RC where there is an op-amp buffer at the start

don't use .param please

-----  
TypeError Traceback (most recent call last)

Cell In[4], line 1

----> 1 lib = build\_library()

3 print(lib.render("rc\_multi", R="1k", C="1n", stages=3))

4 # R1 VIN N1 1k

5 # C1 N1 0 1n

6 # R2 N1 N2 1k

7 # ...

Cell In[3], line 124, in build\_library()

121 lib.register("buffered\_rc\_single", \_make\_buffered\_rc\_single(R="1k", C="1n"))

123 # Dynamic templates receive R/C at render time

--> 124 lib.register("rc\_multi", DynamicNetlistTemplate("rc\_multi", buffered=False))

125 lib.register("buffered\_rc\_multi", DynamicNetlistTemplate("buffered\_rc\_multi", buffered=True))

127 return lib

Cell In[3], line 56, in DynamicNetlistTemplate.\_\_init\_\_(self, name, buffered)

55 def \_\_init\_\_(self, name: str, buffered: bool):

----> 56 super().\_\_init\_\_(name=name, required\_params={"R", "C", "stages"})

57 self.buffered = buffered

TypeError: NetlistTemplate.\_\_init\_\_() missing 1 required positional argument: 'template'

-----  
TypeError Traceback (most recent call last)

Cell In[6], line 1

----> 1 lib = build\_library()

3 print(lib.render("rc\_multi", R="1k", C="1n", stages=3))

4 # R1 VIN N1 1k

5 # C1 N1 0 1n

6 # R2 N1 N2 1k

7 # ...

Cell In[5], line 125, in build\_library()

122 lib = NetlistLibrary()

124 # Static templates bake R/C in at construction time

--> 125 lib.register("rc\_single", \_make\_rc\_single(R="1k", C="1n"))

126 lib.register("buffered\_rc\_single", \_make\_buffered\_rc\_single(R="1k", C="1n"))

128 # Dynamic templates receive R/C at render time

Cell In[5], line 104, in \_make\_rc\_single(R, C)

97 stages = \_rc\_stages(1, R=R, C=C)

98 template = (

99 f"\* Single-stage passive RC low-pass filter\n"

100 f"\* fc = 1 / (2 \* pi \* R \* C)\n\n"

```

101 f"V1 VIN 0 AC 1\n\n"
102 f"{stages}\n"
103)
--> 104 return NetlistTemplate(name="rc_single", template=template)
TypeError: NetlistTemplate.__init__() got an unexpected keyword argument 'template'

```

```

```python
import subprocess
import tempfile
import os
import re

NGSPICE_EXE = r"C:\Users\Luk27182\Downloads\ngspice-46_64\Spice64\bin\ngspice.exe"

def simulate_attenuation(netlist_body: str, frequency: float) -> float | None:
    """
    Run an ngspice AC simulation and return the attenuation at VOUT (in dB).

    Parameters
    -----
    netlist_body : str
        SPICE netlist fragment containing only the circuit elements and .param
        lines (no title line, no .AC/.PRINT/.END). Must define nodes VIN and VOUT.
    frequency : float
        Frequency (Hz) at which to evaluate the gain/attenuation.

    Returns
    -----
    float | None
        Attenuation in dB at the requested frequency, or None on any failure.
    """
    # Basic check: VIN and VOUT must appear somewhere in the netlist body
    body_upper = netlist_body.upper()
    if "VIN" not in body_upper or "VOUT" not in body_upper:
        return None

    netlist = (
        f"* Auto-generated AC simulation\n"
        + netlist_body.strip()
        + f"\n.AC LIN 1 {frequency} {frequency}\n"
        + ".PRINT AC VM(VOUT)\n"
        + ".END\n"
    )

    work_dir = tempfile.gettempdir()
    netlist_path = os.path.join(work_dir, "sim_attn.cir")
    output_path = os.path.join(work_dir, "sim_attn.txt")

    try:
        with open(netlist_path, "w") as f:
            f.write(netlist)
        if os.path.exists(output_path):
            os.unlink(output_path)

```



```

result = subprocess.run(
    [NGSPICE_EXE, "-b", "-o", output_path, netlist_path],
    check=True,
    capture_output=True,
)
except (subprocess.CalledProcessError, FileNotFoundError):
    return None

```

```

try:
    with open(output_path, "rb") as f:
        raw = f.read().decode("utf-8", errors="ignore")
except OSError:
    return None

```

```

# Parse the single data row: index<TAB>freq<TAB>magnitude
for line in raw.splitlines():
    parts = line.strip().split("\t")
    if len(parts) >= 3:
        try:
            mag = float(parts[2])
            import math
            return 20 * math.log10(abs(mag))
        except (ValueError, ZeroDivisionError):
            continue

```

```

return None
...

```

adjust this so that it gets rid of all lines starting with a . before reading in the netlist.

how does the code know what 1n means?

reasonable values for resistor/capacitor?

If I change R by xn and C by $x1/n$, is there any change in the circuit?

```

```python
Define min/max R, min/max C values here.

def create_prompts():
 """
 Takes in:
 # of examples per type (rc_single, etc)

 will
 1. create netlists of each type with randomly sampled R/C values within the range
 2. use find_lowpass_cutoff_frequency (which takes in netlist and returns cutoff frequency as the first output in its output tuple)
 3. samples one point f_c uniform random 1-3 decades below the cutoff freq, and one point f_s uniform random 1-3 decades
 above cutoff freq
 4. simulates attenuation using simulate_attenuation at these frequencies: this returns attenuation. Prompt should add a bit of
 leeway: a maximum atten at fc and a minimum attenuation at fs, which is a bit more "loose" than the measured simulations
 5. samples a random prompt from templates (stored as TEMPLATES: dict[str, list[str]], where the keys are the type of filter and

```

the values are list of possible prompt templates)

6. stores all outputs in ./outputs/lpf\_dataset.json:

make sure to store both the finished prompt, the netlist, the name of topology, the relevant measurements/requirements for checking.

```
"""
```

```
...
```

for reference here is what TEMPLATES is:

```
PS C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new> python .\datagen_new\gen_lowpass.py
```

```
C:\Users\Luk27182\AppData\Local\Programs\Python\Python311\python.exe: can't open file
```

```
'C:\\Users\\Luk27182\\Desktop\\NL2SPICE\\datagen_new\\datagen_new\\gen_lowpass.py': [Errno 2] No such file or directory
```

```
PS C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new>
```

```
PS C:\Users\Luk27182\Desktop\NL2SPICE> python .\datagen_new\gen_lowpass.py
```

```
Traceback (most recent call last):
```

```
File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\gen_lowpass.py", line 8, in <module>
```

```
from .utils.find_cutoffs import find_lowpass_cutoff_frequency
```

```
ImportError: attempted relative import wit
```

where are outputs

```
```python
```

```
def create_prompts(
```

```
n_per_type: int,
```

```
output_path: str = "./outputs/lpf_dataset.json",
```

```
) -> list[dict]:
```

```
"""
```

Parameters

```
-----
```

```
n_per_type : int
```

Number of examples to generate for each topology key in TEMPLATES.

```
lib : NetlistLibrary
```

Pre-built netlist library (from build_library()).

```
output_path : str
```

Where to write the JSON dataset.

Returns

```
-----
```

```
list[dict]
```

The generated records (also written to disk).

```
"""
```

```
Path(output_path).parent.mkdir(parents=True, exist_ok=True)
```

```
records = []
```

```
for topology, prompt_templates in TEMPLATES.items():
```

```
for _ in range(n_per_type):
```

```
# 1. Sample R/C and build netlist
```

```
R_str, C_str = _sample_rc()
```

```
if topology in ("rc_single", "buffered_rc_single"):
```

```
stages = 1
```

```

else:
    stages = random.randint(2, 4)

if topology in ("rc_multi", "buffered_rc_multi"):
    netlist = lib.render(topology, R=R_str, C=C_str, stages=stages)
else:
    netlist = lib.render(topology, R=R_str, C=C_str)

# 2. Find cutoff frequency
fc_hz, *_ = find_lowpass_cutoff_frequency(netlist)
if fc_hz is None:
    continue

# 3. Sample one passband point and one stopband point
decades_below = random.uniform(1, 3)
decades_above = random.uniform(1, 3)
f_pass = fc_hz / (10 ** decades_below)
f_stop = fc_hz * (10 ** decades_above)

# 4. Simulate attenuation at both points
db_pass = simulate_attenuation(netlist, f_pass)
db_stop = simulate_attenuation(netlist, f_stop)
if db_pass is None or db_stop is None:
    continue

pb, atten = _add_leeway(db_pass, db_stop)

# 5. Fill a random prompt template
template_str = random.choice(prompt_templates)
prompt = template_str.format(
    fc = _hz_label(f_pass),
    fs = _hz_label(f_stop),
    pb = pb,
    atten = atten,
)

# 6. Store record
records.append({
    "prompt": prompt,
    "netlist": netlist,
    "topology": topology,
    "stages": stages,
    "R": R_str,
    "C": C_str,
    "fc_hz": fc_hz,
    "f_pass_hz": f_pass,
    "f_stop_hz": f_stop,
    "db_pass": db_pass,
    "db_stop": db_stop,
    "pb_spec_db": pb,
    "atten_spec_db": atten,
})

with open(output_path, "w") as f:

```

```
json.dump(records, f, indent=2)
```

```
print(f"Wrote {len(records)} records to {output_path}")
return records
'''
```

add a tqdm

```
'''python
import subprocess
import tempfile
import os
import re

NGSPICE_EXE = r"C:\Users\Luk27182\Downloads\ngspice-46_64\Spice64\bin\ngspice.exe"

def simulate_attenuation(netlist_body: str, frequency: float) -> float | None:
    """
    Run an ngspice AC simulation and return the attenuation at VOUT (in dB).

    Parameters
    -----
    netlist_body : str
        SPICE netlist fragment containing only the circuit elements and .param
        lines (no title line, no .AC/.PRINT/.END). Must define nodes VIN and VOUT.
    frequency : float
        Frequency (Hz) at which to evaluate the gain/attenuation.

    Returns
    -----
    float | None
        Attenuation in dB at the requested frequency, or None on any failure.
    """
    # Basic check: VIN and VOUT must appear somewhere in the netlist body
    body_upper = netlist_body.upper()
    if "VIN" not in body_upper or "VOUT" not in body_upper:
        return None

    # Strip any dot-commands the caller may have included
    cleaned = "\n".join(
        line for line in netlist_body.splitlines()
        if not line.strip().startswith(".")
    )

    netlist = (
        "** Auto-generated AC simulation\n"
        + cleaned.strip()
        + f"\n.AC LIN 1 {frequency} {frequency}\n"
        + ".PRINT AC VM(VOUT)\n"
        + ".END\n"
    )
```

```

work_dir = tempfile.gettempdir()
netlist_path = os.path.join(work_dir, "sim_attn.cir")
output_path = os.path.join(work_dir, "sim_attn.txt")

try:
    with open(netlist_path, "w") as f:
        f.write(netlist)
    if os.path.exists(output_path):
        os.unlink(output_path)

    result = subprocess.run(
        [NGSPICE_EXE, "-b", "-o", output_path, netlist_path],
        check=True,
        capture_output=True,
    )
except (subprocess.CalledProcessError, FileNotFoundError):
    return None

try:
    with open(output_path, "rb") as f:
        raw = f.read().decode("utf-8", errors="ignore")
    except OSError:
        return None

# Parse the single data row: index<TAB>freq<TAB>magnitude
for line in raw.splitlines():
    parts = line.strip().split("\t")
    if len(parts) >= 3:
        try:
            mag = float(parts[2])
            import math
            return 20 * math.log10(abs(mag))
        except (ValueError, ZeroDivisionError):
            continue

return None
...

```

can you adjust this so that the ngspice window doesn't flash?

the window is still flagging

Low-pass filter design for low-impedance source

```

```json
Create a simple low-pass filter for a low-impedance signal source. The passband should extend to 1.18 Hz with less than 3.0
dB insertion loss. At 533 kHz the circuit should provide at least 54.6 dB of rejection. Minimise cost; no active components are
permitted.
...

```

netlist output: use VIN and VOUT as the names.

## Buffer before RC filter with op amp

what determines if we want a buffer before an RC filter? with op amp

following this format, create prompt templates for

```
```python
buffered_rc_single
```
```

and

```
```python
buffered_rc_multi
```
```

adjust the prompt: never actually say to use an op-amp / need to use a voltage follower . Just say things that imply it is needed: like saying a high impedance source, variable source impedance, decouple the source, resistive driver, etc. You should let the model figure out the op amp is needed all on its own.

```
```python

def create_prompts(
    n_per_type: int,
    output_path: str = "./outputs/lpf_dataset.json",
) -> list[dict]:
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    records = []

    total = n_per_type * len(TEMPLATES)
    pbar = tqdm(total=total, desc="Generating prompts")

    for topology, prompt_templates in TEMPLATES.items():
        for _ in range(n_per_type):
            pbar.set_postfix(topology=topology, records=len(records))

            R_str, C_str = _sample_rc()
            if topology in ("rc_single", "buffered_rc_single"):
                stages = 1
            else:
                stages = random.randint(2, 4)

            if topology in ("rc_multi", "buffered_rc_multi"):
                netlist = lib.render(topology, R=R_str, C=C_str, stages=stages)
            else:
                netlist = lib.render(topology, R=R_str, C=C_str)

            fc_hz, *_ = find_lowpass_cutoff_frequency(netlist)
            if fc_hz is None:
```

```

pbar.update(1)
continue

decades_below = random.uniform(1, 3)
decades_above = random.uniform(1, 3)
f_pass = fc_hz / (10 ** decades_below)
f_stop = fc_hz * (10 ** decades_above)

db_pass = simulate_attenuation(netlist, f_pass)
db_stop = simulate_attenuation(netlist, f_stop)
if db_pass is None or db_stop is None:
    pbar.update(1)
    continue

pb, atten = _add_leeway(db_pass, db_stop)

template_str = random.choice(prompt_templates)
prompt = template_str.format(
    fc = _hz_label(f_pass),
    fs = _hz_label(f_stop),
    pb = pb,
    atten = atten,
)

records.append({
    "prompt": prompt,
    "netlist": netlist,
    "topology": topology,
    "stages": stages,
    "R": R_str,
    "C": C_str,
    "fc_hz": fc_hz,
    "f_pass_hz": f_pass,
    "f_stop_hz": f_stop,
    "db_pass": db_pass,
    "db_stop": db_stop,
    "pb_spec_db": pb,
    "atten_spec_db": atten,
})
pbar.update(1)

pbar.close()

with open(output_path, "w") as f:
    json.dump(records, f, indent=2)

print(f"Wrote {len(records)} records to {output_path}")
return records

...

```

Now adjust this to work with the new prompts

write a System prompt that describes how to do the outputs, and how to do a voltage following op-amp if required. Specify never to use dot commands like .param

make it more general: For designing circuits in general. Don't mention specific topologies.

no but remember don't make it do dot commands like .ac

also in/out commands should be VIN and VOUT.

SPICE netlist generation for analog circuits

```
```markdown
```

```
SYSEYEM_PROMPT = ""
```

You are an expert analog circuit designer. When given a circuit specification, you must produce a SPICE netlist that meets the requirements.

### ## Output Format

Return only the raw SPICE netlist. Do not include any explanation, commentary, markdown formatting, or code fences — just the netlist text itself, ready to be saved as a .cir file.

### ## Netlist Structure

Every netlist must follow this structure, in order:

1. A title line (the very first line, never blank)
2. Component definitions
3. The AC source and ground node
4. The .end statement

### ## Component Guidelines

**\*\*Resistors:\*\*** R<name> <node+> <node-> <value>

Example: R1 VIN mid 10k

**\*\*Capacitors:\*\*** C<name> <node+> <node-> <value>

Use standard suffixes: p, n, u, m, k, meg

Example: C1 mid VOUT 100n

**\*\*Inductors:\*\*** L<name> <node+> <node-> <value>

Example: L1 VIN VOUT 10u

**\*\*AC Voltage Source:\*\*** Always name it Vin, connected between the VIN node and ground (0).

Example: Vin VIN 0 AC 1

**\*\*Ground:\*\*** Always use node 0.

**\*\*Input node:\*\*** Always name it `VIN`.

**\*\*Output node:\*\*** Always name it `VOUT`.

### ## Modelling Active Devices



Never use semiconductor models, subcircuits, or library components. All active behaviour must be modelled using ideal SPICE primitives:

**\*\*Unity-gain buffer / voltage follower:\*\***

Use a unity-gain VCVS (E element):

E<name> <out\_node> 0 <in\_node> 0 1.0

This presents infinite impedance to the driving node and zero output impedance — use it wherever the source impedance must be decoupled from the downstream network, or wherever a low-impedance drive is required.

**\*\*Amplifier with gain G:\*\***

E<name> <out\_node> 0 <in\_node> 0 <G>

Example (gain of 10): Eamp VOUT 0 VIN 0 10.0

**\*\*Differential input:\*\***

E<name> <out\_node> 0 <node+> <node-> <G>

Example: Eamp VOUT 0 vp vn 100k

**\*\*Current-controlled elements and other dependent sources\*\*** (F, G, H) may be used where appropriate, following standard SPICE two-port syntax.

## ## Node Naming

Use short, descriptive lowercase names for internal nodes: n1, n2, mid, buf, vp, vn, etc.

Never reuse node names for different nets.

## ## Forbidden

- Never include any dot commands of any kind — no .ac, .tran, .op, .param, .model, .lib, .subckt, .include, or any other dot command. The only permitted dot statement is .end
- Never leave the first line blank
- Never add comments or semicolons
- Never use semiconductor device primitives (Q, M, J, D) or any model that requires .model or .lib
- Never define active devices as subcircuits; always use dependent source primitives

\*\*\*\*

...

```json

Build a low-frequency pass network to protect a downstream circuit from high-frequency interference. The insertion loss in the passband up to 163 Hz must stay below 3.0 dB. At 186 kHz, the signal level must be reduced by at least 35.9 dB. Restrict the design to resistors and capacitors; use as few as possible.

...

Wiggle room interpretation

how much "wiggle room" is given here?

...

Build a low-frequency pass network to protect a downstream circuit from high-frequency interference. The insertion loss in the passband up to 163 Hz must stay below 3.0 dB. At 186 kHz, the signal level must be reduced by at least 35.9 dB. Restrict the design to resistors and capacitors; use as few as possible.

...

here was one prompt. Here how many decades/db?

how many db per decade

why is it so much leeway?

adjust the end of my code so I can adjust how many per type with a command

Low-pass filter design with RC components

```
```json
```

Design a low-pass filter to clean up a sensor signal. Signals at 1.37 kHz and below must pass with no more than 1.4 dB of attenuation. Interference at 4.6 MHz must be reduced by at least 59.7 dB. Use only resistors and capacitors, and keep the component count as small as possible.

```
```
```

compute db per decade.

Low-pass filter design with steep roll-off

```
```json
```

Design a low-pass filter with a steep roll-off for a signal conditioning application. The passband must extend to 66 Hz with no more than 1.2 dB of insertion loss. Frequencies at 146 kHz must be attenuated by at least 141.7 dB. Use only resistors and capacitors; minimise the total number of components.

```
```
```

High-pass filter cutoff frequency finder

```
```python
```

```
def find_lowpass_cutoff_frequency(
 netlist_body: str,
 tolerance: float = 0.5,
) -> tuple[float | None, plt.Figure | None]:
 """
```

Find the -3dB cutoff frequency of a low-pass filter by binary searching over sampled attenuation values, and return a Bode magnitude plot of all sampled points.

Assumes:

- The filter passes DC (gain  $\approx 0$  dB at 1 Hz)
- Gain rolls off monotonically with frequency (e.g. RC ladder, Butterworth)

Not suitable for high-pass, band-pass, band-stop, or filters with passband ripple (Chebyshev, elliptic).

Parameters

```

```

netlist\_body : str

SPICE netlist fragment passed directly to simulate\_attenuation.

tolerance : float

Stop bisecting when the bracket width is within this fraction of the midpoint frequency. Default 0.01 (1%). Increase for fewer samples and a rougher estimate; decrease for more precision.

Returns

-----

tuple[float | None, plt.Figure | None]

- The -3dB cutoff frequency in Hz, or None on any simulation failure.
- A matplotlib Figure showing the Bode plot, or None on failure.

=====

samples: list[tuple[float, float]] = []

def sample(freq: float) -> float | None:

val = simulate\_attenuation(netlist\_body, frequency=freq)

if val is not None:

samples.append((freq, val))

return val

# Baseline near-DC gain

baseline\_db = sample(1.0)

if baseline\_db is None:

return None, None

target\_db = baseline\_db - 3.0

# Walk up in decades to find an upper bound where gain is below target

upper\_freq = 10.0

for \_ in range(12):

val = sample(upper\_freq)

if val is None:

return None, None

if val < target\_db:

break

upper\_freq \*= 10.0

else:

return None, None

lower\_freq = upper\_freq / 10.0

# Binary search in log-frequency space, exit once bracket is within tolerance

while (upper\_freq - lower\_freq) / (upper\_freq + lower\_freq) \* 2 > tolerance:

mid\_freq = math.exp((math.log(lower\_freq) + math.log(upper\_freq)) / 2)

val = sample(mid\_freq)

if val is None:

return None, None

if val < target\_db:

upper\_freq = mid\_freq

else:

lower\_freq = mid\_freq

cutoff\_freq = math.exp((math.log(lower\_freq) + math.log(upper\_freq)) / 2)

# --- Plot ---

samples.sort(key=lambda x: x[0])

```

freqs, dbs = zip(*samples)

fig, ax = plt.subplots(figsize=(8, 4))
ax.scatter(freqs, dbs, s=18, zorder=5, label="Samples")
ax.plot(freqs, dbs, linewidth=1, alpha=0.6, label="Response")
ax.axvline(
 cutoff_freq,
 color="red",
 linewidth=1,
 linestyle="--",
 label=f"f_c = {cutoff_freq:.1f} Hz",
)
ax.axhline(
 target_db,
 color="gray",
 linewidth=0.8,
 linestyle=":",
 label=f"{target_db:.1f} dB",
)
ax.set_xscale("log")
ax.set_xlabel("Frequency (Hz)")
ax.set_ylabel("Attenuation (dB)")
ax.set_title("Low-pass filter — Bode magnitude plot")
ax.legend()
ax.grid(True, which="both", alpha=0.3)
fig.tight_layout()

return cutoff_freq, fig
'''

```

Do the same thing as this but for high pass

```

'''python
netlist = """
V1 VIN 0 AC 1

R1 VIN N1 1.76k
C1 N1 0 37.6n
R2 N1 N2 1.76k
C2 N2 0 37.6n
R3 N2 N3 1.76k
C3 N3 0 37.6n
R4 N3 VOUT 1.76k
C4 VOUT 0 37.6n
"""
'''

convert this to a highpass

```

## SPICE netlist generation for analog circuits

here is your system prompt:

```markdown

You are an expert analog circuit designer. When given a circuit specification, you must produce a SPICE netlist that meets the requirements.

Output Format

First, reason carefully and thoroughly about the circuit design to ensure it satisfies the specification. Then return only the final SPICE netlist.

The netlist must be placed inside a ```spice``` code block. Do not include any explanation or commentary outside the code block.

Netlist Structure

Every netlist must follow this structure, in order:

1. Component definitions
2. The AC source and ground node
3. The .end statement

Do not include a title line.

Component Guidelines

Resistors: R<name> <node+> <node-> <value>

Example: R1 VIN mid 10k

Capacitors: C<name> <node+> <node-> <value>

Use standard suffixes: p, n, u, m, k, meg

Example: C1 mid VOUT 100n

Inductors: L<name> <node+> <node-> <value>

Example: L1 VIN VOUT 10u

AC Voltage Source: Always name it Vin, connected between the VIN node and ground (0).

Example: Vin VIN 0 AC 1

Ground: Always use node 0.

Input node: Always name it `VIN`.

Output node: Always name it `VOUT`.

Modelling Active Devices

Never use semiconductor models, subcircuits, or library components. All active behaviour must be modelled using ideal SPICE primitives:

Unity-gain buffer / voltage follower:

Use a unity-gain VCVS (E element):

E<name> <out_node> 0 <in_node> 0 1.0

Amplifier with gain G:

```
E<name> <out_node> 0 <in_node> 0 <G>
```

```
**Differential input:**
```

```
E<name> <out_node> 0 <node+> <node-> <G>
```

Current-controlled elements (F, G, H) may be used where appropriate.

Node Naming

Use short, descriptive lowercase names for internal nodes: n1, n2, mid, buf, vp, vn, etc.
Never reuse node names for different nets.

Forbidden

- No title line
- No comments or semicolons
- No dot commands except .end
- No semiconductor primitives (Q, M, J, D)
- No .model, .lib, .subckt, or .include
- No explanations outside the ```spice``` block
- ...

Design a low-pass filter with a steep roll-off for a signal conditioning application. The passband must extend to 66 Hz with no more than 1.2 dB of insertion loss. Frequencies at 146 kHz must be attenuated by at least 141.7 dB. Use only resistors and capacitors; minimise the total number of components.

```
```python
simulate_attenuation(netlist, 66), simulate_attenuation(netlist, 146e3)
```
```

is this what I should measure?

adjust this file to be for HIGH pass insteazd (all prompts)

now adjust this to work with high pass: assume

```
```python
find_highpass_cutoff_frequency
```
```

and

```
```python
prompt_templates.high_pass_templates
```
```

exist

Band pass filter cutoff adjustment

adjust this to work for band pass filters (so return two cutoffs)

```

```python
netlist = """
* 2-stage passive RC high-pass filter buffered (input op-amp buffer)
V1 VIN 0 AC 1

* Input buffer (ideal unity-gain voltage follower)
EBUF VBUF 0 VIN 0 1

C1 VBUF N1 3.35n
R1 N1 0 80.5k
C2 N1 VOUT 3.35n
R2 VOUT 0 80.5k
"""
...

```

adjust to be band pass

adjust these all for band pass. Copy the format, but note that now you will need to stop frequencies to test.

now copy this but for band pass.

## Band pass filter design requirements

band pass fully passive RC filter.

as a netlist

adjust this to use this architecture for the non-buffered. And a similar thing for multi stage.

full code

## Band-pass filter specification validator

```

```json
Design a band-pass filter with steep roll-off on both edges for a signal conditioning application. The passband must span 511 Hz to 5.3 kHz with no more than 5.7 dB of insertion loss. Frequencies at 6.17 Hz and 130 kHz must be attenuated by at least 46.2 dB. Use only resistors and capacitors; minimise the total number of components.
...

```

write a quick test to see if a netlist passes the spec, assuming

```

```python
simulate_attenuation
...

```

exists and returns attenuation (negative val) in db

it failed. Here's the generation code:

why did it fail?

it specifically failed in the passband:

```

```python
def test_bandpass_filter_spec(netlist):
    """
    Verifies a passive RC band-pass filter netlist against:
    Passband : 511 Hz – 5.3 kHz, max insertion loss 5.7 dB
    Stopband : ≤6.17 Hz and ≥130 kHz, min attenuation 46.2 dB

    simulate_attenuation(netlist, freq_hz) -> attenuation in dB (negative = loss)
    """

    PASSBAND_LOW = 511.0 # Hz
    PASSBAND_HIGH = 5_300.0 # Hz
    STOPBAND_LOW = 6.17 # Hz
    STOPBAND_HIGH = 130_000.0 # Hz

    MAX_INSERTION_LOSS_DB = -5.7 # must be >= this (less negative)
    MIN_STOPBAND_ATTEN_DB = -46.2 # must be <= this (more negative)

    failures = []

    # --- Passband check (spot-check at several frequencies) ---
    passband_freqs = [
        PASSBAND_LOW,
        1_000.0,
        2_000.0,
        PASSBAND_HIGH,
    ]
    for f in passband_freqs:
        att = simulate_attenuation(netlist, f)
        if att < MAX_INSERTION_LOSS_DB:
            failures.append(
                f"FAIL passband @ {f:.1f} Hz: "
                f"{att:.2f} dB (limit {MAX_INSERTION_LOSS_DB} dB)"
            )

    # --- Stopband checks ---
    for f in (STOPBAND_LOW, STOPBAND_HIGH):
        att = simulate_attenuation(netlist, f)
        if att > MIN_STOPBAND_ATTEN_DB:
            failures.append(
                f"FAIL stopband @ {f:.2f} Hz: "
                f"{att:.2f} dB (need <= {MIN_STOPBAND_ATTEN_DB} dB)"
            )

    # --- Report ---
    if failures:
        print(f"FAIL — {len(failures)} violation(s):")
        for msg in failures:
            print(f" {msg}")

```



```
return False
```

```
print("PASS — all constraints met")
return True
'''
```

adjust this so it actually takes in a whole json from the output of that py file (so it can access all the tests/netlists)

```
'''json
{
  "prompt": "Provide a band-pass filter design for a general-purpose noise-reduction application. The passband should span 281
  Hz to 1.51 kHz with no more than 4.1 dB loss. At 15.1 Hz and 674 kHz the filter must deliver >= 25.1 dB of attenuation. No
  active components; keep the design as simple and cheap as possible.",
  "netlist": "** Single-stage passive RC band-pass filter\nV1 VIN 0 AC 1\n\n* High-pass section (sets lower cutoff)\nCH1 VIN NH1
  51.9\n\nRH1 NH1 0 10.1k\n\n* Low-pass section (sets upper cutoff)\nRL1 NH1 VOUT 16.3k\n\nCL1 VOUT 0 8.67n\n",
  "topology": "rc_single",
  "stages": 1,
  "R_hp": "10.1k",
  "C_hp": "51.9n",
  "R_lp": "16.3k",
  "C_lp": "8.67n",
  "f_low_hz": 215.48600380158467,
  "f_high_hz": 1880.2132997659812,
  "f_pass_hz": 586.07313065668,
  "f_stop_low_hz": 15.06915314117944,
  "f_stop_high_hz": 674422.5272272214,
  "db_pass": -3.147036507762411,
  "db_stop_low": -26.101134188437133,
  "db_stop_high": -55.5463935644184,
  "pb_spec_db": 4.1,
  "atten_spec_db": 25.1
},
'''
```

records look like this.

just run the tests in the record pass/fail

```
[1] FAIL rc_single s=1
passband @ 215.5 Hz: -5.50 dB (limit -4.1 dB)
passband @ 1880.2 Hz: -6.37 dB (limit -4.1 dB)
[2] FAIL rc_multi s=2
passband @ 237.0 Hz: -7.28 dB (limit -5.7 dB)
passband @ 6324.9 Hz: -8.44 dB (limit -5.7 dB)
[3] FAIL buffered_rc_single s=1
passband @ 226.8 Hz: -5.27 dB (limit -4.9 dB)
[4] FAIL buffered_rc_multi s=2
passband @ 2818.0 Hz: -5.20 dB (limit -3.3 dB)
passband @ 22503.0 Hz: -5.96 dB (limit -3.3 dB)
0/4 passed
```

Only test at

```
```json
"f_pass_hz": 586.07313065668,
"f_stop_low_hz": 15.06915314117944,
"f_stop_high_hz": 674422.5272272214,
```
```

full coode for the gen_bandpass.py

Building relays from scratch

can I make my own relays from scratch? how?

what about a capacitor?

what about an LED

easiest way to get a visible measure of there being a voltage: from scratch

at 5V .

what about like an electromagnet moving something to have the top be either red or blue for example

why does a relay have 8 pins?

what level of capacitor do I need for RAM in a relay circuit?

Making copper wire from raw copper

how to make copper wire from raw copper metal

Multi-core generation optimization

make it so it generates things in multiple cores: as many as possible

(hope) C:\Users\Luk27182\Desktop\NL2SPICE>python ./datagen_new/gen_bandpass.py -n 50

Generating prompts (24 workers): 0% | 0/240 [00:03<?, ?it/s]

multiprocessing.pool.RemoteTraceback:

Traceback (most recent call last):

File "c:\Users\Luk27182\miniconda3\envs\hope\Lib\multiprocessing\pool.py", line 125, in worker

result = (True, func(*args, **kws))

~~~~~^

File "c:\Users\Luk27182\miniconda3\envs\hope\Lib\multiprocessing\pool.py", line 48, in mapstar

return list(map(\*args))

File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen\_new\gen\_bandpass.py", line 236, in \_generate\_one

result = find\_bandpass\_cutoff\_frequencies(netlist)

```

File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\utils\find_cutoffs.py", line 284, in
find_bandpass_cutoff_frequencies
val = sample(freq)
File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\utils\find_cutoffs.py", line 272, in sample
val = simulate_attenuation(netlist_body, frequency=freq)
File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\utils\measure_atten.py", line 53, in simulate_attenuation
os.unlink(output_path)
~~~~~
PermissionError: [WinError 32] The process cannot access the file because it is being used by another process:
'C:\Users\Luk27182\AppData\Local\Temp\sim_attn.txt'
~~~~~

The above exception was the direct cause of the following exception:
Traceback (most recent call last):
File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\gen_bandpass.py", line 353, in <module>
create_prompts(args.n_per_type, args.output, args.num_workers)
~~~~~
File "C:\Users\Luk27182\Desktop\NL2SPICE\datagen_new\gen_bandpass.py", line 322, in create_prompts
for record in pool.imap_unordered(_generate_one, jobs, chunksize=4):
~~~~~
File "c:\Users\Luk27182\miniconda3\envs\hope\Lib\multiprocessing\pool.py", line 451, in <genexpr>
return (item for chunk in result for item in chunk)
^^^^^^
File "c:\Users\Luk27182\miniconda3\envs\hope\Lib\multiprocessing\pool.py", line 873, in next
raise value
PermissionError: [Errno 13] The process cannot access the file because it is being used by another process:
'C:\Users\Luk27182\AppData\Local\Temp\sim_attn.txt'
(hope) C:\Users\Luk27182\Desktop\NL2SPICE>

```

```

```python
import subprocess
import tempfile
import os
import re

NGSPICE_EXE = r"C:\Users\Luk27182\Downloads\ngspice-46_64\Spice64\bin\ngspice.exe"

def simulate_attenuation(netlist_body: str, frequency: float) -> float | None:
    """
    Run an ngspice AC simulation and return the attenuation at VOUT (in dB).

    Parameters
    -----
    netlist_body : str
        SPICE netlist fragment containing only the circuit elements and .param
        lines (no title line, no .AC/.PRINT/.END). Must define nodes VIN and VOUT.
    frequency : float
        Frequency (Hz) at which to evaluate the gain/attenuation.

    Returns
    -----
    float | None
        Attenuation in dB at the requested frequency, or None on any failure.
    """
    # Basic check: VIN and VOUT must appear somewhere in the netlist body

```

```

body_upper = netlist_body.upper()
if "VIN" not in body_upper or "VOUT" not in body_upper:
    return None

# Strip any dot-commands the caller may have included
cleaned = "\n".join(
    line for line in netlist_body.splitlines()
    if not line.strip().startswith(".")
)

netlist = (
    "** Auto-generated AC simulation\n"
    + cleaned.strip()
    + f"\n.AC LIN 1 {frequency} {frequency}\n"
    + ".PRINT AC VM(VOUT)\n"
    + ".END\n"
)

work_dir = tempfile.gettempdir()
netlist_path = os.path.join(work_dir, "sim_attn.cir")
output_path = os.path.join(work_dir, "sim_attn.txt")

try:
    with open(netlist_path, "w") as f:
        f.write(netlist)
    if os.path.exists(output_path):
        os.unlink(output_path)

    si = subprocess.STARTUPINFO()
    si.dwFlags |= subprocess.STARTF_USESHOWWINDOW
    si.wShowWindow = subprocess.SW_HIDE

    result = subprocess.run(
        [NGSPICE_EXE, "-b", "-o", output_path, netlist_path],
        check=True,
        capture_output=True,
        creationflags=subprocess.CREATE_NO_WINDOW,
        startupinfo=si,
    )
except (subprocess.CalledProcessError, FileNotFoundError):
    return None

try:
    with open(output_path, "rb") as f:
        raw = f.read().decode("utf-8", errors="ignore")
    except OSError:
        return None

# Parse the single data row: index<TAB>freq<TAB>magnitude
for line in raw.splitlines():
    parts = line.strip().split("\t")
    if len(parts) >= 3:
        try:

```

```
mag = float(parts[2])
import math
return 20 * math.log10(abs(mag))
except (ValueError, ZeroDivisionError):
continue

return None
'''
```

heres the py file- give me a correction (that won't break other references to the function)

fix so maximum of 16 threads.

the tqdm bar isn't updating

## Notch filter implementation

make this but for notch filters.

Describe four topologies: single stage RC, multistage RC, and buffering with a op amp at the start before single stage and multistage

convert these prompt templates into these four notch filters (from band pass)

now convert this to do notch. Assume datagen\_new\prompt\_templates\notch\_templates.py exists as does

```
```python
find_notch_cutoff_frequencies
'''
```

does this include the buffered multi stage?

full code

## AnalogCoder problem generation pipeline

<https://github.com/laiyao1/AnalogCoder/tree/main> in this repo, where is the complete pipeline for generating the problems?

look through each of them: see if ANY prompts require specific numbers for resistors/capacitors.

here I gotch you! Tell me which files you want to see (all of them one by one)

You aim to design a topology for a given circuit described in the text.

Please ensure your designed circuit topology works properly and achieves the design requirements.

To make the task easier, I provide you with some existing subcircuits you can directly use by calling functions in Python with the PySpice library.

Now I would like you to help me design a complex analog circuit based on them.

Here is an example:

```
## Question
```

Design an opamp with 470 ohm resistance load.

Input node name: in

Output node name: out

You can directly use the following subcircuits.

### Subcircuits Info

Id	Circuit Type	Gain/Differential-mode gain (dB)	Common-mode gain (dB)	Input	Output
- Opamp	20	-10	Vin	Vout	

### Call Info

To use them, please insert the following codes.

```
```python
from example_lib import *
# declare the subcircuit
circuit.subcircuit('BasicOperationalAmplifier())
# create a subcircuit instance
circuit.X('1', 'BasicOperationalAmplifier', 'Vin', 'Vout')
```

## Answer
```python
from PySpice.Spice.Netlist import Circuit
from PySpice.Unit import *
from example_lib import *
circuit = Circuit('Operational Amplifier')
# Define the MOSFET models
circuit.model('nmos_model', 'nmos', level=1, kp=100e-6, vto=0.5)
circuit.model('pmos_model', 'pmos', level=1, kp=50e-6, vto=-0.5)
circuit.V('input', 'in', circuit.gnd, 2.5@u_V)
circuit.subcircuit(BasicOperationalAmplifier())
circuit.X('op', 'BasicOperationalAmplifier', 'in', circuit.gnd, 'out')
R = 470
circuit.R('load', 'out', circuit.gnd, R)
simulator = circuit.simulator()
```
```

As you have seen, the output of your designed topology should be in a complete Python code, describing the topology of integrated analog circuits according to the design plan.

Please make sure your Python code is compatible with PySpice.

Please give the runnable code without any placeholders.

Do not write other redundant codes after ``simulator = circuit.simulator()``.

There are some tips you should remember all the time:

1. For the MOSFET definition `circuit.MOSFET(name, drain, gate, source, bulk, model, w=w1,l=l1)`, be careful about the parameter sequence.
2. You should connect the bulk of a MOSFET to its source.
3. Please use the MOSFET threshold voltage, when setting the bias voltage.
4. Avoid giving any AC voltage in the sources, just consider the operating points.
5. Make sure the input and output node names appear in the circuit.
6. Assume the  $V_{dd} = 5.0$  V. Do not need to add the power supply for subcircuits.

## Question

Design [TASK].

Input node name: [INPUT].

Output node name: [OUTPUT].

You can directly use the following subcircuits.

### Subcircuits Info

[SUBCIRCUITS\_INFO]

#### Note

[NOTE\_INFO]

### Call Info

To use them, please insert the following codes.

[CALL\_INFO]

## Answer

problem set tsv:

I have the following implemented subcircuits you can directly call them by Python code with the Pyspice library, we list all basic information on them.

| Id | Type          | Circuit                                                                                                                                                                                       | Gain (dB) | Common Mode Gain (dB) | # of inputs | # of outputs | Input Phase              |
|----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------------------|-------------|--------------|--------------------------|
| 1  | Amplifier     | a single-stage common-source amplifier with resistive load R                                                                                                                                  | 17.03     | NA                    | 1           | 1            | inverting                |
| 2  | Amplifier     | A three-stage amplifier with single input and output (each stage is common-source with resistive load)                                                                                        | 44.13     | NA                    | 1           | 1            | inverting                |
| 3  | Amplifier     | A common-drain amplifier (a.k.a. a source follower) with resistive load R (output Vout at the source)                                                                                         | -1.15     | NA                    | 1           | 1            | non-inverting            |
| 4  | Amplifier     | A single-stage common-gate amplifier with resistive load R (input signal Vin must be applied at the source terminal)                                                                          | 17.03     | NA                    | 1           | 1            | non-inverting            |
| 5  | Amplifier     | A single-stage cascode amplifier with two NMOS transistors provides a single-ended output through a resistive load R                                                                          | 24.08     | NA                    | 1           | 1            | inverting                |
| 6  | Inverter      | A NMOS inverter with resistive load R                                                                                                                                                         | NA        | NA                    | 1           | 1            | NA                       |
| 7  | Inverter      | A logical inverter with 1 NMOS and 1 PMOS                                                                                                                                                     | NA        | NA                    | 1           | 1            | NA                       |
| 8  | CurrentMirror | A simple NMOS constant current source with resistive load R                                                                                                                                   | NA        | NA                    | 1           | 1            | NA                       |
| 9  | Amplifier     | A two-stage amplifier with a Miller compensation capacitor                                                                                                                                    | 75.94     | NA                    | 1           | 1            | -90 degree               |
| 10 | Amplifier     | A single-stage amplifier (common-source with PMOS diode-connected load (gate and drain are shorted))                                                                                          | 3.01      | NA                    | 1           | 1            | inverting                |
| 11 | Opamp         | A differential opamp with an active PMOS current mirror load, a tail current source, and two outputs                                                                                          | 200       | -40.13                | 2           | 2            | non-inverting, inverting |
| 12 | CurrentMirror | A cascode current mirror with 4 mosfets (2 stacked at input side with diode-connected, 2 stacked at output side), reference current source input Iref (connected to Vdd) and resistive load R | NA        | NA                    | 1           | 1            | NA                       |
| 13 | Opamp         | A single-stage differential common-source opamp with dual resistive loads, tail current, and a single output                                                                                  | 24.04     | 21.59                 | 2           | 1            | non-inverting, inverting |
| 14 | Opamp         | A two-stage differential opamp (first stage: common-source with an active load and a tail current, second stage: common-source with an active load)                                           | 150.36    | -58.72                | 2           | 2            | non-inverting, inverting |
| 15 | Opamp         | A single-stage telescopic cascode opamp with two outputs (4 nmos as cascode input pair, 4 pmos as cascode loads, and 1 tail current)                                                          | 41.62     | -135.67               | 2           | 2            | non-inverting, inverting |

Now, you need to design [TASK]. Please maximize the design success rate, thus the circuit with two inputs and the highest possible gain has greater flexibility. Please choose the subcircuits from the above table you will use and make the number of chosen subcircuits as few as possible.

Please give out the IDs of the subcircuits that you choose, and enumerate them in a Python list like ``[0]``.

retrieval prompt

I have the following implemented subcircuits you can directly call them by Python code with the Pyspice library, we list all basic information on them.

| Id | Type      | Circuit                                                                                                              | Gain (dB) | Common Mode Gain (dB) | # of inputs | # of outputs | Input Phase   |
|----|-----------|----------------------------------------------------------------------------------------------------------------------|-----------|-----------------------|-------------|--------------|---------------|
| 1  | Amplifier | a single-stage common-source amplifier with resistive load R                                                         | 17.03     | NA                    | 1           | 1            | inverting     |
| 2  | Amplifier | A three-stage amplifier with single input and output (each stage is common-source with resistive load)               | 44.13     | NA                    | 1           | 1            | inverting     |
| 3  | Amplifier | A common-drain amplifier (a.k.a. a source follower) with resistive load R (output Vout at the source)                | -1.15     | NA                    | 1           | 1            | non-inverting |
| 4  | Amplifier | A single-stage common-gate amplifier with resistive load R (input signal Vin must be applied at the source terminal) | 17.03     | NA                    | 1           | 1            | non-inverting |
| 5  | Amplifier | A single-stage cascode amplifier with two NMOS transistors provides a single-ended output through a resistive        |           |                       |             |              |               |

load R | 24.08 | NA | 1 | 1 | inverting |  
 | 6 | Inverter | A NMOS inverter with resistive load R | NA | NA | 1 | 1 | NA |  
 | 7 | Inverter | A logical inverter with 1 NMOS and 1 PMOS | NA | NA | 1 | 1 | NA |  
 | 8 | CurrentMirror | A simple NMOS constant current source with resistive load R | NA | NA | 1 | 1 | NA |  
 | 9 | Amplifier | A two-stage amplifier with a Miller compensation capacitor | 75.94 | NA | 1 | 1 | -90 degree |  
 | 10 | Amplifier | A single-stage amplifier (common-source with PMOS diode-connected load (gate and drain are shorted)) | 3.01 | NA | 1 | 1 | inverting |  
 | 11 | Opamp | A differential opamp with an active PMOS current mirror load, a tail current source, and two outputs | 200 | -40.13 | 2 | 2 | non-inverting, inverting |  
 | 12 | CurrentMirror | A cascode current mirror with 4 mosfets (2 stacked at input side with diode-connected, 2 stacked at output side), reference current source input Iref (connected to Vdd) and resistive load R | NA | NA | 1 | 1 | NA |  
 | 13 | Opamp | A single-stage differential common-source opamp with dual resistive loads, tail current, and a single output | 24.04 | 21.59 | 2 | 1 | non-inverting, inverting |  
 | 14 | Opamp | A two-stage differential opamp (first stage: common-source with an active load and a tail current, second stage: common-source with an active load) | 150.36 | -58.72 | 2 | 2 | non-inverting, inverting |  
 | 15 | Opamp | A single-stage telescopic cascode opamp with two outputs (4 nmos as cascode input pair, 4 pmos as cascode loads, and 1 tail current) | 41.62 | -135.67 | 2 | 2 | non-inverting, inverting |

Now, you need to design [TASK]. Please maximize the design success rate, thus the circuit with two inputs and the highest possible gain has greater flexibility. Please choose the subcircuits from the above table you will use and make the number of chosen subcircuits as few as possible.

Please give out the IDs of the subcircuits that you choose, and enumerate them in a Python list like `""[0]""`.

problem set tsv

Id Level Circuit Input Output Type Submodule Name

|    |        |                                                                                                                                                                                               |                                                        |               |                                  |
|----|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|----------------------------------|
| 1  | Easy   | a single-stage common-source amplifier with resistive load R                                                                                                                                  | Vin Vout                                               | Amplifier     | SingleStageAmp                   |
| 2  | Easy   | A three-stage amplifier with single input and output (each stage is common-source with resistive load)                                                                                        | Vin Vout                                               | Amplifier     | ThreeStageAmp                    |
| 3  | Easy   | A common-drain amplifier (a.k.a. a source follower) with resistive load R (output Vout at the source)                                                                                         | Vin Vout                                               | Amplifier     | CommonDrainAmp                   |
| 4  | Easy   | A single-stage common-gate amplifier with resistive load R (input signal Vin must be applied at the source terminal)                                                                          | Vin, Vbias Vout                                        | Amplifier     | CommonGateAmp                    |
| 5  | Easy   | A single-stage cascode amplifier with two NMOS transistors provides a single-ended output through a resistive load R                                                                          | Vin, Vbias Vout                                        | Amplifier     | SingleStageCascodeAmp            |
| 6  | Easy   | A NMOS inverter with resistive load R                                                                                                                                                         | Vin Vout                                               | Inverter      | NMOSInverter                     |
| 7  | Easy   | A logical inverter with 1 NMOS and 1 PMOS                                                                                                                                                     | Vin Vout                                               | Inverter      | LogicalInverter                  |
| 8  | Easy   | A simple NMOS constant current source with resistive load R                                                                                                                                   | Vbias Vout                                             | CurrentMirror | NMOSConstantCurrentSource        |
| 9  | Medium | A two-stage amplifier with a Miller compensation capacitor                                                                                                                                    | Vin Vout                                               | Amplifier     | TwoStageOpampMiller              |
| 10 | Medium | A single-stage amplifier (common-source with PMOS diode-connected load (gate and drain are shorted))                                                                                          | Vin Vout                                               | Amplifier     | CommonSourceAmpDiodeLoad         |
| 11 | Medium | A differential opamp with an active PMOS current mirror load, a tail current source, and two outputs                                                                                          | Vinp, Vinn, Vbias Voutp, Vout                          | Opamp         | SingleStageOpamp                 |
| 12 | Medium | A cascode current mirror with 4 mosfets (2 stacked at input side with diode-connected, 2 stacked at output side), reference current source input Iref (connected to Vdd) and resistive load R | Iref Iout                                              | CurrentMirror | CascodeCurrentMirror             |
| 13 | Medium | A single-stage differential common-source opamp with dual resistive loads, tail current, and a single output                                                                                  | Vinp, Vinn Vout                                        | Opamp         | SingleStageDiffCommonSourceOpamp |
| 14 | Hard   | A two-stage differential opamp (first stage: common-source with an active load and a tail current, second stage: common-source with an active load)                                           | Vinp, Vinn, Vbias1, Vbias2, Vbias3 Voutp, Vout         | Opamp         | OpampResistanceLoad              |
| 15 | Hard   | A single-stage telescopic cascode opamp with two outputs (4 nmos as cascode input pair, 4 pmos as cascode loads, and 1 tail current)                                                          | Vinp, Vinn, Vbias1, Vbias2, Vbias3, Vbias4 Voutp, Vout | Opamp         | SingleStageDiffOpamp             |
| 16 | Hard   | An RC phase-shift oscillator                                                                                                                                                                  | -                                                      | Oscillator    | Unknown                          |
| 17 | Hard   | A Wien Bridge oscillator                                                                                                                                                                      | -                                                      | Oscillator    | Unknown                          |
| 18 | Hard   | An Opamp integrator with resistor R1 and capacitor Cf                                                                                                                                         | Vin Vout                                               | Integrator    | Unknown                          |



19 Hard An Opamp differentiator with resistor Rf and capacitor C1 Vin Vout Differentiator Unknown  
 20 Hard An Opamp adder to make  $V_{out}=-(V_{in1}+V_{in2})$  Vin1, Vin2 Vout Adder Unknown  
 21 Hard An Op-amp subtractor to make  $V_{out}=V_{in2}-V_{in1}$  Vin1, Vin2 Vout Subtractor Unknown  
 22 Hard A non-inverting Schmitt trigger with positive feedback op-map Vin Vout Schmitt Unknown  
 23 Hard A voltage-controlled oscillator Vin Vout VCO Unknown  
 24 Hard A phase-locked loop clk\_ref clk\_p PLL Unknown

lib info tsv:

| Id | Type          | Av (dB)             | Com                 | Av (dB)       | Vin(n) | Phase | Voltage | Bias |
|----|---------------|---------------------|---------------------|---------------|--------|-------|---------|------|
| 0  | Amplifier     | 17.025166965695615  | NA                  | inverting     | 1.21   |       |         |      |
| 1  | CurrentMirror | NA                  | NA                  | NA            | 0.0    |       |         |      |
| 2  | Inverter      | NA                  | NA                  | inverting     | 1.0    |       |         |      |
| 3  | Inverter      | NA                  | NA                  | inverting     | 0.0    |       |         |      |
| 4  | Amplifier     | -1.1483743996327227 | NA                  | non-inverting | 3.71   |       |         |      |
| 5  | Amplifier     | 17.02516696691898   | NA                  | non-inverting | 0.79   |       |         |      |
| 6  | Amplifier     | 3.0102999372276096  | NA                  | inverting     | 1.97   |       |         |      |
| 7  | Amplifier     | 24.082399647689847  | NA                  | inverting     | 0.82   |       |         |      |
| 8  | Amplifier     | 75.94026898353593   | NA                  | -90 degree    | 1.0    |       |         |      |
| 9  | Opamp         | 199.99956997487047  | -40.12867535462482  | inverting     | 2.49   |       |         |      |
| 10 | CurrentMirror | NA                  | NA                  | NA            | 0.0    |       |         |      |
| 11 | Opamp         | 41.61900557997383   | -135.66721120644925 | inverting     | 1.79   |       |         |      |
| 12 | Opamp         | 150.36128137602688  | -58.71904858948874  | inverting     | 1.65   |       |         |      |
| 13 | Opamp         | 24.044031388496503  | 21.58664528028884   | inverting     | 0.87   |       |         |      |
| 14 | Amplifier     | 44.128945502128325  | NA                  | inverting     | 1.65   |       |         |      |

<https://arxiv.org/pdf/2508.02518> read this paper for the pro version- see if the pro version fixes this

are the circuit types always meant to be fixed (like what logically requires what) or is the idea that the model learns the subcircuits by itself

why don't humans do bayes searches with a PC? They do math

is bayes search less efficient than an LLM with intuition?

how are previous modules brought into the context?

## Documenting a RAG retrieval function

```
```python
# Package Versions:
# -----
# sentence_transformers 5.4.1
# transformers 5.6.2
# huggingface_hub 1.12.0
# torch 2.9.0+cpu
# numpy 2.4.1
# tqdm 4.67.1
# pathlib built-in (Python 3.4+)

from sentence_transformers import SentenceTransformer
import torch
```

```

import numpy as np
from pathlib import Path

explainer_files = {}

# Get the directory where the current script/notebook is located
current_dir = Path(__file__).parent if '__file__' in globals() else Path.cwd()

# Look for RAG_docs folder in the same location
rag_docs_path = current_dir / "RAG_docs"

for file_path in rag_docs_path.glob("*.md"):
    name = file_path.stem # Gets filename without extension
    with open(file_path, encoding='utf-8') as f:
        text = f.read()
        explainer_files[name] = text

query_model = SentenceTransformer("best_query_model")
doc_model = SentenceTransformer("best_doc_model")

def run_custom(prompt, verbose=False):
    query_model.eval()
    doc_model.eval()

    doc_names = list(explainer_files.keys())
    doc_texts = [explainer_files[k] for k in doc_names]

    with torch.no_grad():
        docs_emb = doc_model.encode(doc_texts, show_progress_bar=verbose, convert_to_numpy=True)
        docs_emb = docs_emb / np.linalg.norm(docs_emb, axis=1, keepdims=True)

    with torch.no_grad():
        query_emb = query_model.encode(prompt, convert_to_numpy=True)
        query_emb = query_emb / np.linalg.norm(query_emb)

    scores = docs_emb @ query_emb
    top_idx = np.argmax(scores)
    predicted_doc = doc_names[top_idx]

    return {"detected_topology": predicted_doc, "full_RAG_system_prompt":
        explainer_files["base_prompt"]+explainer_files[predicted_doc]}
    ...

add in better function definition and comments to make it clear how to use

```

## Easiest Way to Fine-Tune RAG

easiest way to fine tune a rag model? I have examples.

I want to fine tune an embedder (small)

then how to get predictions?

how many epochs to train for if I have 4000 pairs of queries to docs (4000 queries, 16 docs). Starting top 1 accuracy is 0.25.

0 epochs: 0.231. 1 epoch: 0.266

how to adjust code?

```
train_examples = []
```

for example in RAG\_data:

```
train_examples.append(InputExample(texts=[example["prompt"],
example["explainer_content"]]))
```

```
train_dataloader = DataLoader(train_examples, shuffle=True, batch_size=128)
```

```
print("dataset prepared")
```

```
model.fit(
train_objectives=[(train_dataloader, train_loss)],
epochs=1,
warmup_steps=100
)
```

## RAG in Hybrid Pipeline

We aim to use a two-stage pipeline, using both retrieval augmented generate (RAG) and agentic simulations. For the RAG component, we will first create a set of documents describing common circuit design patterns generated using a foundation model such as GPT 5 from a subset of the Bhandari et al. [2025] dataset. These design patterns will describe the key aspects of the design and relevant equations without focusing on the specific numerical values of the problem. We will then fine tune a document encoder that can embed natural language specifications in similar semantic directions to embedded design pattern documents, allowing our NL2SPICE agent to retrieve relevant design pattern documents at the start of generation. For the structured reasoning with agentic simulations component, we will use a chain-of-thought prompt along with a finetuned model to complete relevant circuit mathematics, selecting a topology, and run experiments via spice simulations in ngspice.

Does this really use rag?